Experiment No. 1

Title: Inter-process communication

Aim– Program to demonstrate Inter-Process communication using Python

Theory: Inter Process Communication is a type of mechanism usually provided by the operating system (OS). The main goal of this mechanism is to provide communications in between several processes. It is defined as, "Inter-process communication is used for exchanging useful information between numerous threads in one or more processes (or programs)."

Some of the different approaches for Inter-process communication are:

● **Sockets -** It acts as a type of endpoint for receiving or sending the data in a network. It is correct for data sent between processes on the same computer or data sent between different computers on the same network. Hence, it is used by several types of operating systems.

● **Files -** A file is a type of data record or a document stored on the disk and can be acquired on demand by the file server. Another most important thing is that several processes can access that file as required or needed.

● **Signals -** They are the messages of systems that are sent by one process to another. Usually, they are not used to send the data but to remote commands in between several processes.

Need for IPC:

● It helps to speedup modularity

● Computational

● Privilege separation

● Convenience

● Helps operating systems to communicate with each other and synchronize their actions.

Code:

Steps to perform Inter-process Communication in Python using Client and Server programs are as follows

Steps 1– Write Server Program IPCServer.py.

```
"""
Experiment 1: Inter-Process Communication (IPC)
IPC Server using Python Sockets
Run this FIRST, then run exp1_ipc_client.py in another terminal.
"""
import socket
def main():
    print("\n **** INTERPROCESS COMMUNICATION ****\n")
    print("\n *** SERVER PROCESS STARTED ***\n")
```

```
    PORT = 1200
    print(f"\n * SERVER IS READY AND WAITING TO RECEIVE DATA FROM CLIENT PROCESS
ON PORT {PORT}")

    server_socket = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
    server_socket.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
    server_socket.bind(('localhost', PORT))
    server_socket.listen(1)

    client_socket, client_address = server_socket.accept()
    print(f"\n * Client is connected with IP address {client_address[0]} and port Number
{client_address[1]}")

    # Receive two integers from client
    data = client_socket.recv(1024).decode()
    a, b = map(int, data.split(','))

    print("\n SERVER RECEIVED")
    print(f"\n Number 1 ====>{a}")
    print(f"\n Number 2 ====>{b}")

    c = a + b
    client_socket.send(str(c).encode())
    print(f"\n SERVER PROCESS HAS EXECUTED REQUESTED PROCESS AND SENT RESULT
{c} TO THE CLIENT \n")

    client_socket.close()
    print("\n SERVER PROCESS EXITING.........")
    server_socket.close()
if __name__ == "__main__":
    main()
```

Step 2– Write Client Program IPCClient.py.

```
"""
Experiment 1: Inter-Process Communication (IPC)
IPC Client using Python Sockets
```

```
Run this AFTER starting exp1_ipc_server.py in another terminal.
"""
import socket
def main():
    print("\n\t************* CLIENT PROCESS STARTED *********************")
    print("\n ******** PLEASE ENTER THE VALUES OF Number 1 AND Number 2 TO PASS THEM
TO SERVER PROCESS******** \n")

    a = int(input("Enter Number 1: "))
    print(f"Number 1 ====>{a}")

    b = int(input("Enter Number 2: "))
    print(f"Number 2 ====>{b}")

    client_socket = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
    client_socket.connect(('localhost', 1200))

    # Send both numbers as comma-separated string
    client_socket.send(f"{a},{b}".encode())

    result_data = client_socket.recv(1024).decode()
    result = int(result_data)

    print("\n.............CLIENT PROCESS HAS RECEIVED RESULT FROM SERVER.............\n")
    print(f"\n THE ADDITION OF {a} AND {b} IS {result}")

    client_socket.close()
if __name__ == "__main__":
    main()
```

Output-

1) Run Server program

```
PROBLEMS 2   OUTPUT   DEBUG CONSOLE   TERMINAL   PORTS   SPELL CHECKER 2

Microsoft Windows [Version 10.0.26200.7705]
(c) Microsoft Corporation. All rights reserved.

C:\Users\DELL\Desktop\dc 1-5>python ipcserver.py

 **** INTERPROCESS COMMUNICATION ****


 *** SERVER PROCESS STARTED ***


 * SERVER IS READY AND WAITING TO RECEIVE DATA FROM CLIENT PROCESS ON PORT 1200
```

2) Run Client Program and Enter two numbers

```
PROBLEMS 2   OUTPUT   DEBUG CONSOLE   TERMINAL   PORTS   SPELL CHECKER 2

C:\Users\DELL\Desktop\dc 1-5>python ipcclient.py

        ************* CLIENT PROCESS STARTED *********************

 ********* PLEASE ENTER THE VALUES OF Number 1 AND Number 2 TO PASS THEM TO SERVER PROCESS********

Enter Number 1:
```

3) Execution from Client side

```
C:\Users\DELL\Desktop\dc 1-5>python ipcclient.py

        ************* CLIENT PROCESS STARTED **********************

 ********* PLEASE ENTER THE VALUES OF Number 1 AND Number 2 TO PASS THEM TO SERVER PROCESS********

Enter Number 1: 21
Number 1 ====>21
Enter Number 2: 16
Number 2 ====>16

.............CLIENT PROCESS HAS RECEIVED RESULT FROM SERVER...............


 THE ADDITION OF 21 AND 16 IS 37

C:\Users\DELL\Desktop\dc 1-5>
```

4) Execution from Server side

```
C:\Users\DELL\Desktop\dc 1-5>python ipcserver.py

 **** INTERPROCESS COMMUNICATION ****


 *** SERVER PROCESS STARTED ***


 * SERVER IS READY AND WAITING TO RECEIVE DATA FROM CLIENT PROCESS ON PORT 1200

 * Client is connected with IP address 127.0.0.1 and port Number 57277

 SERVER RECEIVED

 Number 1 ====>21

 Number 2 ====>16

 SERVER PROCESS HAS EXECUTED REQUESTED PROCESS AND SENT RESULT 37 TO THE CLIENT


 SERVER PROCESS EXITING.........

C:\Users\DELL\Desktop\dc 1-5>
```

Experiment No. 2

Title: Client/Server using RPC/RMI

Aim: Program to demonstrate Client/Server application Using RPC

Theory:

RPC (Remote Procedure Call)

RPC (Remote Procedure Call) is a protocol that allows a program to request a service from a program located on another computer in a network. Python provides RPC functionality through its built-in **xmlrpc** module. The **xmlrpc** module allows a client to call functions on a remote server as if they were local functions, providing remote communication between applications using two components — **Client Proxy** and **Server Handler**.

Client Proxy (Stub)

The Client Proxy is an object created using xmlrpc.client.ServerProxy, acts as a gateway for the client side. All the outgoing requests are routed through it. It resides at the client side and represents the remote server. When the caller invokes a method on the proxy object, it does the following tasks:

- It initiates a connection with the remote server over HTTP
- It serializes the method name and parameters into XML format and transmits them to the remote server
- It waits for the result
- It reads the return value or exception from the XML response, and
- It finally, returns the value to the caller

Server Handler (Skeleton)

The Server Handler is an object created using SimpleXMLRPCServer, acts as a gateway for the server-side object. All the incoming requests are routed through it. When the Server Handler receives the incoming request, it does the following tasks:

- It reads and parses the XML request to extract the method name and parameters
- It invokes the registered Python function on the actual server with those parameters, and
- It serializes the result back into XML format and transmits it to the caller

The RPC Application is composed of two programs:

1. **Server Program** — which contains the registration of functions using server.register_function() to bind the functions that will be called by the client, equivalent to Naming.rebind() in Java RMI.
2. **Client Program** — which contains the method calling using xmlrpc.client.ServerProxy to locate and connect to the server, equivalent to Naming.lookup() in Java RMI, and then directly calls the remote method on the proxy object.

The following steps will explain the RPC program to add two numbers:

Step 1 – Write the Server Program exp2_rpc_server.py which registers the add() function using server.register_function() and starts listening for client requests on port 8000.

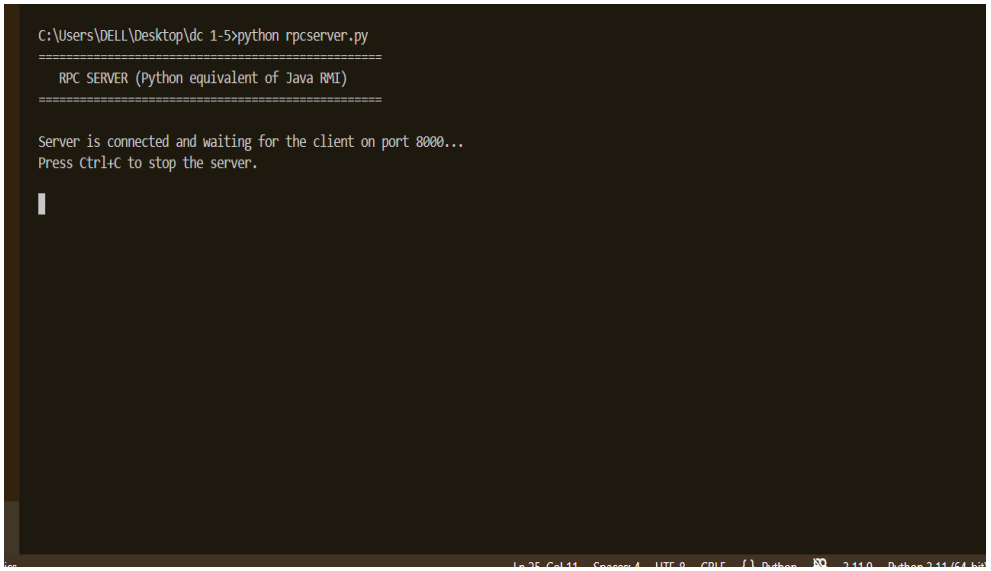
```
from xmlrpc.server import SimpleXMLRPCServer
def add(n1, n2):
    """Remote method: adds two numbers and returns the result."""
    print(f"  [Server] Received request to add {n1} + {n2}")
    result = n1 + n2
    print(f"  [Server] Returning result: {result}")
    return result

def main():
    HOST = 'localhost'
    PORT = 8000
    server = SimpleXMLRPCServer((HOST, PORT), logRequests=False)
    server.register_function(add, 'add')   # Register the remote method

    print("=" * 50)
    print("   RPC SERVER (Python equivalent of Java RMI)")
    print("=" * 50)
    print(f"\nServer is connected and waiting for the client on port {PORT}...")
    print("Press Ctrl+C to stop the server.\n")

    try:
        server.serve_forever()
    except KeyboardInterrupt:
        print("\nServer shutting down.")

if __name__ == "__main__":
    main()
```

Step 2 – Write the Client Program exp2_rpc_client.py which creates a ServerProxy object connecting to http://localhost:8000/, then calls the remote add(10, 2) method through the proxy and prints the result.

```
import xmlrpc.client
def main():
```

```
    print("=" * 50)
    print("   RPC CLIENT (Python equivalent of Java RMI)")
    print("=" * 50)

    # Connect to the remote server (like Naming.lookup in Java RMI)
    proxy = xmlrpc.client.ServerProxy("http://localhost:8000/")

    # Call the remote method 'add' (like ai.sum(10, 2) in Java RMI)
    n1 = 10
    n2 = 2
    print(f"\nCalling remote method: add({n1}, {n2})")
    result = proxy.add(n1, n2)
    print(f"The sum of {n1} and {n2} is: {result}")
if __name__ == "__main__":
    main()
```

Step 3 – Run the Server Program first in one terminal so it starts listening for connections.



Step 4 – Run the Client Program in a second terminal. The client proxy connects to the server, the remote add() method is invoked, and the result is returned and displayed on the client side.

```
C:\Users\DELL\Desktop\dc 1-5>python rpcclient.py
==================================================
   RPC CLIENT (Python equivalent of Java RMI)
==================================================

Calling remote method: add(10, 2)
The sum of 10 and 2 is: 12

C:\Users\DELL\Desktop\dc 1-5>
```

```
C:\Users\DELL\Desktop\dc 1-5>python rpcserver.py
==============================================
   RPC SERVER (Python equivalent of Java RMI)
==============================================

Server is connected and waiting for the client on port 8000...
Press Ctrl+C to stop the server.

  [Server] Received request to add 10 + 2
  [Server] Returning result: 12
```

Experiment No. 3

Title: Group communication

Aim– To implement group communication using Python

Theory:

Group Communication

Group Communication is a type of communication in a distributed system where a message sent by one process is delivered to a group of processes simultaneously. It is widely used in distributed computing to coordinate multiple processes and ensure consistent message delivery across all members of a group.

Group Communication is defined as, "A communication model in which a single message transmitted by one process is received by all the processes that are members of the group." It is an extension of point-to-point communication where instead of one sender and one receiver, there is one sender and multiple receivers.

Key Components of Group Communication

Server

The server acts as a central handler or coordinator in the group communication system. It does not participate in the actual message exchange but instead manages the group by performing the following tasks:

- It listens for incoming client connections on a specific port
- It maintains a list of all active clients (group members) connected to it
- When it receives a message from any client, it broadcasts that message to all other clients in the group
- It handles each client connection in a separate thread to allow simultaneous communication

Master (Sender)

The master is a special client that has the privilege of sending messages to the group. It performs the following tasks:

- It connects to the server and registers itself with a name starting with "master"
- It accepts messages from the user through the keyboard
- It sends those messages to the server, which then broadcasts them to all group members
- It also receives and displays messages broadcast by the server

Slave (Receiver)

The slave is a client that can only receive messages from the group. It performs the following tasks:

- It connects to the server and registers itself with a name such as "slave 1" or "slave 2"
- It continuously listens for incoming messages broadcast by the server
- It displays the received messages on the screen
- It cannot send messages to the group — it is a read-only member

This Application has two important programs, namely– client and server. The client application is copied thrice and given names as Master, slave1 and slave2.The master program is responsible for broadcasting the message among their group while the slave can read those messages. The server is only a handler, which binds multiple clients together into a group. The steps of implementation are as follows:

Program:

Step 1– Write Server Program Server.py

```
import socket
import threading

clients = []   # list of (conn, name) tuples
lock = threading.Lock()

def broadcast(message, sender_name):
    """Send message to all connected clients."""
    with lock:
        for conn, name in clients:
            try:
                conn.send(f"MESSAGE {sender_name}: {message}".encode())
            except:
                pass

def handle_client(conn, addr):
    """Handle an individual client connection."""
    try:
        conn.send("SUBMITNAME".encode())
        name = conn.recv(1024).decode().strip()
        print(f"{name} joined")

        with lock:
            clients.append((conn, name))

        while True:
            data = conn.recv(1024)
            if not data:
                break
            message = data.decode().strip()
            broadcast(message, name)

    except Exception as e:
        print(f"Error: {e}")
    finally:
        with lock:
            clients[:] = [(c, n) for c, n in clients if c != conn]
        conn.close()

def main():
    PORT = 9001
    server = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
    server.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
    server.bind(('localhost', PORT))
```

```
    server.listen(10)
    print(f"The server is running at port {PORT}.")

    try:
        while True:
            conn, addr = server.accept()
            thread = threading.Thread(target=handle_client, args=(conn, addr))
            thread.daemon = True
            thread.start()
    except KeyboardInterrupt:
        print("\nServer shutting down.")
    finally:
        server.close()

if __name__ == "__main__":
    main()
```

Step 2– Write Master Client Program master.py

```
import socket
import threading

def receive_messages(sock):
    """Continuously receive and print messages from server."""
    while True:
        try:
            data = sock.recv(1024)
            if not data:
                break
            message = data.decode().strip()
            if message.startswith("MESSAGE"):
                print(message[8:])  # strip "MESSAGE " prefix
        except:
            break

def main():
    sock = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
    sock.connect(('localhost', 9001))

    # Get name from server prompt
    prompt = sock.recv(1024).decode().strip()
    if prompt == "SUBMITNAME":
        name = input("Enter your name: ").strip()
        sock.send(name.encode())

    print(f"\n*** Connected as '{name}' ***\n")

    # Start background thread to receive messages
```

```
    recv_thread = threading.Thread(target=receive_messages, args=(sock,))
    recv_thread.daemon = True
    recv_thread.start()

    # Master can send messages
    if name.startswith("master"):
        while True:
            try:
                msg = input("Enter a message: ")
                sock.send(msg.encode())
            except (KeyboardInterrupt, EOFError):
                break
    else:
        print("You are a slave - you can only receive messages.")
        recv_thread.join()

    sock.close()

if __name__ == "__main__":
    main()
```

Step 3– Write slave Client Program slave.py (slave1 and slave2)

```
import socket
import threading

def receive_messages(sock):
    """Continuously receive and print messages from server."""
    while True:
        try:
            data = sock.recv(1024)
            if not data:
                break
            message = data.decode().strip()
            if message.startswith("MESSAGE"):
                print(message[8:])  # strip "MESSAGE " prefix
        except:
            break

def main():
    sock = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
    sock.connect(('localhost', 9001))

    prompt = sock.recv(1024).decode().strip()
    if prompt == "SUBMITNAME":
        name = input("Enter your name: ").strip()
```

```
        sock.send(name.encode())

    print(f"\n*** Connected as '{name}' - waiting for messages... ***\n")

    # Slave only receives
    recv_thread = threading.Thread(target=receive_messages, args=(sock,))
    recv_thread.daemon = True
    recv_thread.start()

    try:
        recv_thread.join()
    except KeyboardInterrupt:
        pass

    sock.close()

if __name__ == "__main__":
    main()
```

Output-

Run Server program-

```
PROBLEMS 6   OUTPUT   DEBUG CONSOLE   TERMINAL   PORTS   SPELL CHECKER 6

Microsoft Windows [Version 10.0.26200.7705]
(c) Microsoft Corporation. All rights reserved.

C:\Users\DELL\Desktop\dc 1-5>python groupserver.py
The server is running at port 9001.
```

Run Master Program-

```
C:\Users\DELL\Desktop\dc 1-5>python master.py
Enter your name: master

*** Connected as 'master' ***

Enter a message:
```

Run Slave program-

```
C:\Users\DELL\Desktop\dc 1-5>python slave.py
Enter your name: slave 1

*** Connected as 'slave 1' - waiting for messages... ***
```

```
C:\Users\DELL\Desktop\dc 1-5>python slave.py
Enter your name: slave 2

*** Connected as 'slave 2' - waiting for messages... ***
```

Group Server displays everyone who is connected-

```
C:\Users\DELL\Desktop\dc 1-5>python groupserver.py
The server is running at port 9001.
master joined
slave 1 joined
slave 2 joined
```

Master sends a message to the group-

```
C:\Users\DELL\Desktop\dc 1-5>python master.py
Enter your name: master

*** Connected as 'master' ***

Enter a message: hello , welcome to the group!
master: hello , welcome to the group!
Enter a message:
```

Slaves receive the message sent by the master-

```
C:\Users\DELL\Desktop\dc 1-5>python slave.py
Enter your name: slave 1

*** Connected as 'slave 1' - waiting for messages... ***

master: hello , welcome to the group!
```

```
C:\Users\DELL\Desktop\dc 1-5>python slave.py
Enter your name: slave 2

*** Connected as 'slave 2' - waiting for messages... ***

master: hello , welcome to the group!
```

Experiment No. 4

Title: Clock Synchronization algorithms

Aim– To implement Lamport's Clock Synchronization Algorithm.

Theory:

The algorithm of Lamport timestamps is a simple algorithm used to determine the order of events in a distributed computer system. As different nodes or processes will typically not be perfectly synchronized, this algorithm is used to provide a partial ordering of events with minimal overhead, and conceptually provide a starting point for the more advanced vector clock method. They are named after their creator, Leslie Lamport. Distributed algorithms such as resource synchronization often depend on some method of ordering events to function. For example, consider a system with two processes and a disk. The processes send messages to each other, and also send messages to the disk requesting access. The disk grants access in the order the messages were sent.

For example, process A sends a message to the disk requesting write access, and then sends a read instruction message to process B. Process B receives the message, and as a result sends its own read request message to the disk. If there is a timing delay causing the disk to receive both messages at the same time, it can determine which message happened-before the other: (AA happens-before B B if one can get from AA to B B by a sequence of moves of two types: moving forward while remaining in the same process, and following a message from its sending to its reception.) A logical clock algorithm provides a mechanism to determine facts about the order of such events.

Lamport invented a simple mechanism by which the happened-before ordering can be captured numerically. A Lamport logical clock is an incrementing software counter maintained in each process. Conceptually, this logical clock can be thought of as a clock that only has meaning in relation to messages moving between processes. When a process receives a message, it resynchronizes its logical clock with that sender. The above mentioned vector clock is a generalization of the idea into the context of an arbitrary number of parallel, independent processes. The algorithm follows some simple rules:

1. A process increments its counter before each event in that process;

2. When a process sends a message, it includes its counter value with the message;

3. On receiving a message, the counter of the recipient is updated, if necessary, to the greater of its current counter and the timestamp in the received message.

<u>Algorithm:</u>

- **Happened before relation** (->): a -> b, means 'a' happened before 'b'.
- **Logical Clock:** The criteria for the logical clocks are:
    - [C1]: Ci (a) < Ci(b), [Ci -> Logical Clock, If 'a' happened before 'b', then time of 'a' will be less than 'b' in a particular process.]
    - [C2]: Ci(a) < Cj(b), [Clock value of Ci(a) is less than Cj(b)]

<u>Reference:</u>

- **Process:** Pi

- **Event:** Eij, where i is the process in number and j: jth event in the ith process.
- **tm:** vector time span for message m.

- Ci vector clock associated with process Pi, the jth element is Ci[j] and contains Pi's latest value for the current time in process Pj.

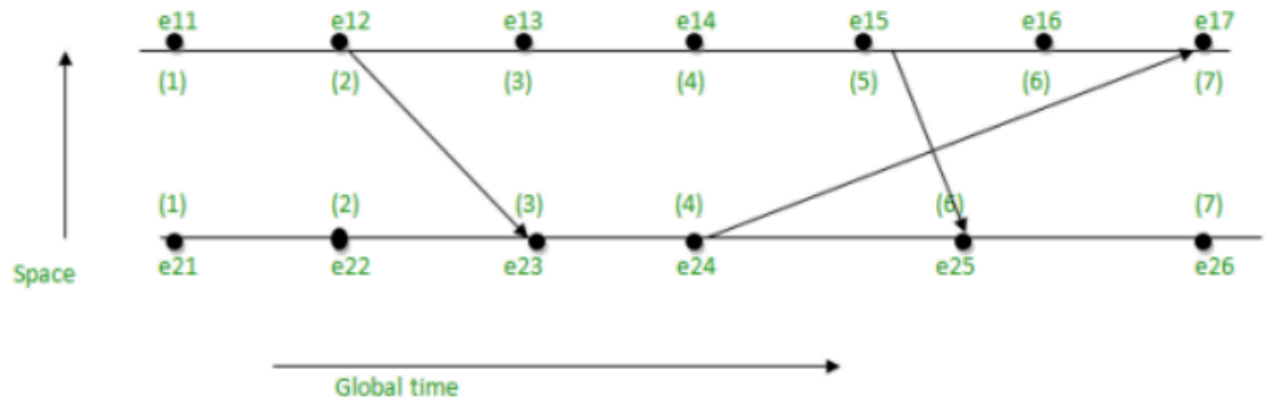
- **d:** drift time, generally d is 1.

Implementation Rules[IR]:

- [IR1]: If a -> b ['a' happened before 'b' within the same process] then, Ci(b) =Ci(a) + d

- [IR2]: Cj = max(Cj, tm + d) [If there's more number of processes, then tm = value of Ci(a), Cj = max value between Cj and tm + d].

For Example:



- Take the starting value as 1, since it is the 1st event and there is no incoming value at the starting point:

  - e11 = 1
  - e21 = 1

- The value of the next point will go on increasing by d (d = 1), if there is no incoming value i.e., to follow [IR1].

  - e12 = e11 + d = 1 + 1 = 2
  - e13 = e12 + d = 2 + 1 = 3
  - e14 = e13 + d = 3 + 1 = 4
  - e15 = e14 + d = 4 + 1 = 5
  - e16 = e15 + d = 5 + 1 = 6
  - e22 = e21 + d = 1 + 1 = 2
  - e24 = e23 + d = 3 + 1 = 4
  - e26 = e25 + d = 6 + 1 = 7

● When there will be an incoming value, then follow [IR2] i.e., take the maximum value between Cj and Tm + d.

- e17 = max(7, 5) = 7, [e16 + d = 6 + 1 = 7, e24 + d = 4 + 1 = 5, maximum among 7 and 5 is 7]
- e23 = max(3, 3) = 3, [e22 + d = 2 + 1 = 3, e12 + d = 2 + 1 = 3, maximum among 3 and 3 is 3]
- e25 = max(5, 6) = 6, [e24 + 1 = 4 + 1 = 5, e15 + d = 5 + 1 = 6, maximum among 5 and 6 is 6]

Program:

```
"""
Theory:
- Each process has a logical clock counter.
- [IR1]: If event 'a' happens before 'b' in same process: C(b) = C(a) + 1
- [IR2]: On receiving a message: C(receiver) = max(C(receiver), C(sender)) + 1

The matrix m[i][j]:
   1  = message sent FROM process P1 event i  TO process P2 event j
  -1  = message received BY P1 at event i FROM P2 at event j
   0  = no message
"""


def max1(a, b):
    """Return the greater of two values."""
    return a if a > b else b


def display(e1, e2, p1, p2):
    """Display the logical timestamps for both processes."""
    print("\nThe time stamps of events in P1:")
    print(" ".join(str(p1[i]) for i in range(e1)))

    print("\nThe time stamps of events in P2:")
    print(" ".join(str(p2[i]) for i in range(e2)))


def lamport_logical_clock(e1, e2, m):
    """
    Compute Lamport timestamps.
```

```
    e1: number of events in P1
    e2: number of events in P2
    m : 2D matrix of size e1 x e2 (message relations)
    """
    # Initialize clocks
    p1 = [i + 1 for i in range(e1)]
    p2 = [i + 1 for i in range(e2)]

    # Print the message matrix
    print("\nMessage Matrix (rows=P1 events, cols=P2 events):")
    header = "\t" + "\t".join(f"e2{j+1}" for j in range(e2))
    print(header)
    for i in range(e1):
        row = f"e1{i+1}\t" + "\t".join(str(m[i][j]) for j in range(e2))
        print(row)

    # Apply Lamport rules
    for i in range(e1):
        for j in range(e2):
            if m[i][j] == 1:
                # P1 sends to P2: update P2's clock [IR2]
                p2[j] = max1(p2[j], p1[i] + 1)
                for k in range(j + 1, e2):
                    p2[k] = p2[k - 1] + 1

            if m[i][j] == -1:
                # P1 receives from P2: update P1's clock [IR2]
                p1[i] = max1(p1[i], p2[j] + 1)
                for k in range(i + 1, e1):
                    p1[k] = p1[k - 1] + 1

    display(e1, e2, p1, p2)

def main():
    # Example: P1 has 5 events, P2 has 3 events
```

```
    e1 = 5
    e2 = 3
    # Message matrix:
    # m[1][2] = 1  => P1's 2nd event sends to P2's 3rd event
    # m[4][1] = -1 => P1's 5th event receives from P2's 2nd event
    m = [
        [0, 0, 0],
        [0, 0, 1],
        [0, 0, 0],
        [0, 0, 0],
        [0, -1, 0],
    ]

    print("=" * 50)
    print("   LAMPORT'S LOGICAL CLOCK ALGORITHM")
    print("=" * 50)
    lamport_logical_clock(e1, e2, m)

if __name__ == "__main__":
    main()
```

Output:

```
C:\Users\DELL\Desktop\dc 1-5>python clock.py
==================================================
   LAMPORT'S LOGICAL CLOCK ALGORITHM
==================================================

Message Matrix (rows=P1 events, cols=P2 events):
        e21     e22     e23
e11     0       0       0
e12     0       0       1
e13     0       0       0
e14     0       0       0
e15     0       -1      0

The time stamps of events in P1:
1 2 3 4 5

The time stamps of events in P2:
1 2 3

C:\Users\DELL\Desktop\dc 1-5>
```

Experiment No. 5

Title: Election Algorithm.

Aim– To implement To Implement Bully Election Algorithms.

Theory: The bully algorithm is a type of Election algorithm which is mainly used for choosing a coordinate. In a distributed system, we need some election algorithms such as bully and ring to get a coordinator that performs functions needed by other processes. Election algorithms select a single process from the processes that act as coordinator. A new process is selected when the selected coordinator process crashes due to some reasons. In order to determine the position where the new copy of coordinator should be restarted, the election

algorithms are used. It assumes that each process has a unique priority number in the system, so the highest priority process will be chosen first as a new coordinator. When the current use coordinator process crashes, it elects a new process having the highest priority number. We note that priority number and pass it to each active process in the distributed system.

The Bully election algorithm is as follows:

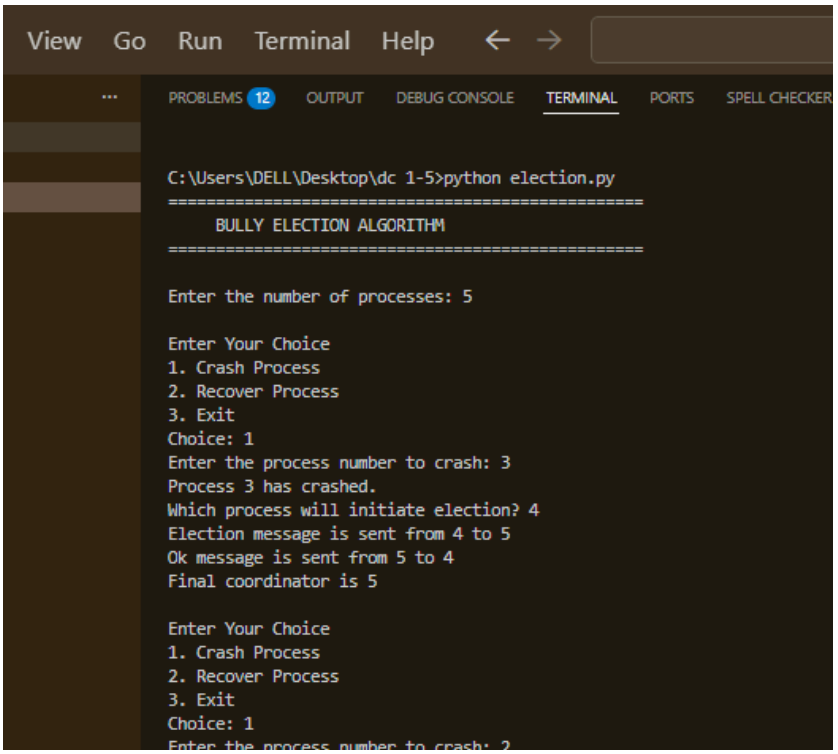
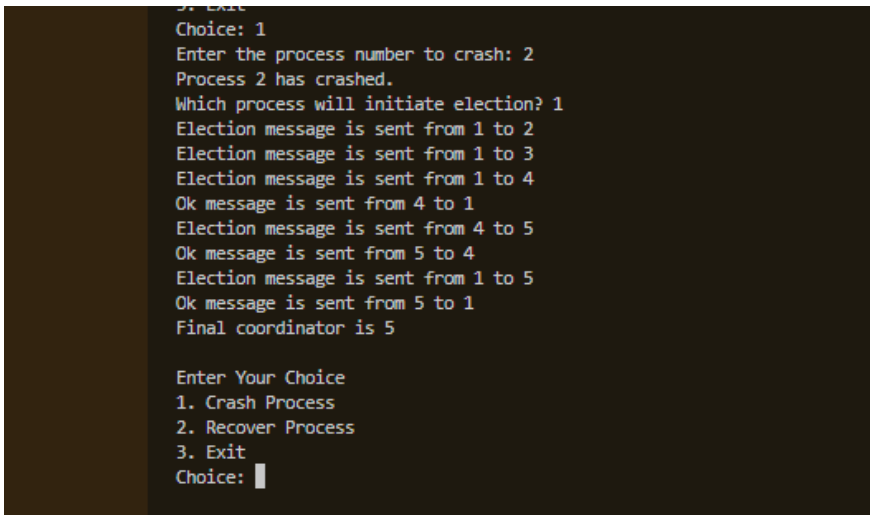
Let's assume that P is a process that sends a message to the coordinator.

1) It will assume that the coordinator process is failed when it doesn't receive any response from the coordinator within the time interval T.
2) An election message will be sent to all the active processes by process P along with the highest priority number.
3) If it will not receive any response within the time interval T, the current process P elects itself as a coordinator.
4) After selecting itself as a coordinator, it again sends a message that process P is elected as their new coordinator to all the processes having lower priority.
5) If process P will receive any response from another process Q within time T:
    a. It again waits for time T to receive another response, i.e., it has been elected as coordinator from process Q.
    b. If it doesn't receive any response within time T, it is assumed to have failed, and the algorithm is restarted.

Program:

```
"""Theory:
- Each process has a unique priority (ID).
- When coordinator crashes, a new election is triggered.
- The process with the highest ID/priority among active processes wins.
- Process sends election messages to all higher-priority processes.
- If no response, it declares itself coordinator.
- If it gets a response, it waits for the higher process to become coordinator."""

MAX_PROCESSES = 100

n = 0
pro = list(range(MAX_PROCESSES))   # pro[i] = i (priority = process index)
sta = [0] * MAX_PROCESSES          # sta[i] = 1 if active, 0 if crashed
```

```
co = 0                          # current coordinator

def elect(ele):
    """Run bully election starting from process 'ele' (1-indexed)."""
    global co
    ele = ele - 1          # convert to 0-indexed
    co = ele + 1           # tentatively set coordinator to this process

    for i in range(n):
        if pro[ele] < pro[i]:
            print(f"Election message is sent from {ele + 1} to {i + 1}")
            if sta[i] == 1:
                print(f"Ok message is sent from {i + 1} to {ele + 1}")
            if sta[i] == 1:
                elect(i + 1)   # higher active process takes over

def main():
    global n, sta, co

    print("=" * 50)
    print("     BULLY ELECTION ALGORITHM")
    print("=" * 50)

    n = int(input("\nEnter the number of processes: "))
    for i in range(n):
        sta[i] = 1
        pro[i] = i

    choice = True
    while choice:
        print("\nEnter Your Choice")
        print("1. Crash Process")
        print("2. Recover Process")
        print("3. Exit")
        ch = int(input("Choice: "))

        cl = 1
        if ch == 1:
            c = int(input("Enter the process number to crash: "))
            sta[c - 1] = 0
            print(f"Process {c} has crashed.")

        elif ch == 2:
            c = int(input("Enter the process number to recover: "))
            sta[c - 1] = 1
            print(f"Process {c} has recovered.")

        elif ch == 3:
```

```
        choice = False
        cl = 0

    else:
        print("Invalid choice.")
        cl = 0

    if cl == 1:
        ele = int(input("Which process will initiate election? "))
        elect(ele)
        print(f"Final coordinator is {co}")

if __name__ == "__main__":
    main()
```

Output: